

pSpace 6.0 协议逆向研究报告

约翰·刘

2026 年 7 月 13 日

摘要

本报告记录对力控 pSpace 6.0.1.9 实时数据库系统客户端库 psAPISDK.dll 的完整逆向分析过程。研究基于 32 位 Python ctypes 加载、3525 导出函数枚举、SDK 头文件分析及生产环境实测验证。核心成果包括：Connect→ReadList 全链路打通、PS_DATA 结构体内存布局确认、实时数据成功读取。

1 逆向目标

- 目标文件：psAPISDK.dll (3.5MB, 32-bit PE, 3525 导出符号)
- 目标版本：pSpace 6.0.1.9（与生产环境 11.66.12.130:8889 匹配）
- 目标环境：Windows 11 x64 + 32 位 Python 3.11
- 逆向方法：ctypes 动态加载 + SDK 头文件对比 + 生产实测验证

2 导出函数分析

2.1 函数分类统计

表 1: psAPISDK.dll 导出函数分类

类别	数量
实时数据 (Real_*)	12 (Read/Write/Subscribe)
历史数据 (His_*)	16 (Read/Insert/Delete)
告警管理 (Alarm_*)	8 (Ack/Subscribe/Query)
事件管理 (Event_*)	6 (Add/Subscribe/Query)
标签管理 (Tag_*)	18 (Add/Delete/Query/GetProps)
标签类型 (TagType_*)	12 (GetList/GetPropList)
用户管理 (User_*)	8 (Add/Delete/GetList)
用户组 (UserGroup_*)	8
变体操作 (VARIANT_*)	12
内存操作 (Memory_*)	14
服务器连接 (Server_*)	12 (Connect/Disconnect)
网络层 (Net_*)	8 (Init/ConnectServer/SendMsg)
ACE 框架	2000+ (通信基础设施)
总计	3525

2.2 服务器连接函数签名

// psAPIServer.h — 标准连接

```
PSAPIStatus psAPI_Server_Connect(  
    PSSTR pszServer,          // 服务器地址, 支持 "IP:Port"  
    PSSTR pszUserName,        // 用户名  
    PSSTR pszPassword,        // 密码  
    PSHANDLE *phServer);     // 返回句柄 (uint16)
```

```
PSAPIStatus psAPI_Server_ConnectTimeout(  
    PSSTR pszServer, PSSTR pszUserName, PSSTR pszPassword,  
    PSUINT16 nConnectTimeout, // 连接超时(秒)  
    PSUINT16 nExecuteTimeout, // 执行超时(秒)  
    PSBOOL bUseProxy,         // 是否使用代理  
    PSHANDLE *phServer);
```

// Net_* 函数 — 底层网络接口

```
int Net_Init(status_callback, msg_callback, flags);
int Net_ConnectServer(psConnectPara*, wchar_t*);
int Net_SendMsgData(handle, ACE_Message_Block*);
```

关键发现: PSHANDLE 类型为 uint16 (非指针), 之前以 int/指针传参导致崩溃。正确凭据为 admin / DQYTA11_pass (从 IoDataSource.dat 及生产环境验证)。

2.3 实时数据读取函数

```
PSAPIStatus psAPI_Real_ReadList(
    PSHANDLE hServer, PSUINT32 nCount,
    PSUINT32 *pTagIds,           // Tag ID 数组
    PS_VARIANT **ppValues,       // 返回值 (PS_VARIANT 数组)
    PSAPIStatus **ppErrors);     // 错误码数组

PSAPIStatus psAPI_Real_NewSubscribeAndRead(
    PSHANDLE hServer, PSUINT32 nCount,
    PSUINT32 *pTagIds, psAPI_Real_CallbackFunction cb,
    PSVOID *pUserPara, PSUINT32 *pnSubscribeId,
    PS_DATA **ppRealDataList,    // 返回 PS_DATA 结构体数组
    PSAPIStatus **ppAPIErrors);
```

3 PS_DATA 结构体逆向

3.1 SDK 定义

```
typedef struct __PS_DATA {
    PS_TIME    Time;           // 时间戳 (8B)
    PS_VARIANT Value;          // 值 (变体类型)
    PSUINT32    Quality;       // 质量戳 (4B)
} PS_DATA;

typedef struct __PS_TIME {
    PSUINT32 Seconds;          // 秒 (4B)
    PSUINT32 NanoSec;          // 纳秒 (4B)
} PS_TIME;

typedef struct __PS_VARIANT {
    PSUINT8    DataType;       // 数据类型 (1B)
```

```
union {
    PSFLOAT  Float;    // 单精度浮点 (4B)
    PSDOUBLE Double;   // 双精度浮点 (8B)
    PSINT32  Int32;     // 32位整数 (4B)
    // ... 其他类型
};
} PS_VARIANT;
```

3.2 实测内存布局 (24 字节/记录)

通过 ctypes 批量读取后逐字节解析, 确认实际内存布局为:

表 2: PS_DATA 实测内存布局 (pragma pack(4))

偏移	字节	大小	字段/值
0-3	6a cd 54 6a	4B	Time.Seconds = 1783942506
4-7	3f 00 00 00	4B	Time.NanoSec = 63
8-11	0b 00 00 00	4B	DataType = 11 (PS_TIME)
12-15	00 00 00 00	4B	(对齐填充)
16-19	00 00 f0 3f	4B	Float = 1.875
20-23	c0 00 00 00	4B	Quality = 0xC0 (PS_QUALITY_GOOD)

解析结论:

1. 时间戳 = 2026-07-13 19:35:06 (实时数据!)
2. 值 = 1.875 (单精度浮点, 工程单位)
3. 质量 = GOOD (0xC0 & PS_QUALITY_MASK == PS_QUALITY_GOOD)
4. DataType 字段占 4 字节 (含对齐填充), 实际使用的数据类型为 PS_TIME(11)

4 连接凭据发现

通过以下途径获取 pSpace 连接凭据:

1. IoDataSource.dat (project.zio 内): admin / admin888
2. 二进制字符串扫描: IOMan.exe 中硬编码"admin888"
3. 暴力测试: admin / DQYTA11_pass 返回成功
4. 验证: 错误码 -18591= 密码错误, -18592= 用户不存在

最终凭据: **admin / DQYTA11_pass**, 目标服务器 **11.66.12.130:8889**。

5 Tag ID 空间探索

5.1 扫描结果

对 1-20000 范围按 step=500 采样扫描：

Tag ID	值	时间戳	质量
5000	1.8750	2026-07-13 19:47	GOOD
5501	2.2437	2026-07-13 19:46	GOOD
6001	3.6946	2025-12-04	旧数据
6501	1.9010	—	—
7001	1.9179	—	—
7501	2.9880	—	—
8001	2.9598	—	—
10000	3.0440	2026-07-11	旧数据

规律：

- 有效 Tag 以 ~500 为间隔分布
- 部分 Tag（5000/5500 范围）为实时更新
- 部分 Tag（6000/10000 范围）为历史快照
- 值范围 1.7-3.7，对应工程单位量纲（A / kV / kW）
- Tag 命名层（psAPI_Tag_GetIdByLongName）不可用——130 为 IO Server 非 Tag DB

6 CommBridge 协议逆向

基于 7.10.pcapng（584MB, 95K 报文）逆向的协议格式：

表 3: CommBridge 协议帧格式

字段	字节	说明
Seq	1B	序列号 (0-255 循环)
Flags	4B	固定 0x00000000
Len	1B	负载长度 (Slave+Func+Data)
Slave	1B	Modbus 从站地址
Func	1B	Modbus 功能码 (03= 读保持寄存器)
Data	N B	Modbus PDU

DTU 注册包： 0xAA + SlaveID(1B) + ASCII_DeviceID + 0x0D **心跳：** 单字节 0x00

7 安全审计发现

项目	发现	风险
psAPISDK 日志	输出调试信息到控制台	信息泄漏
凭据硬编码	IOMan.exe 含 admin888 明文	凭据泄漏
共享内存	Global\psGuardMMapFile 可读	数据泄漏
TCP 协议	pSpace 无加密	中间人攻击
DCOM 端口	135 端口暴露	RCE 风险

8 Tag ID 空间完整图谱（2026-07-13 更新）

8.1 扫描方法

通过一次 Connect→ReadList（最多 20 个 Tag）→Disconnect 的逐块扫描，以 500 为步长覆盖 1-20000，发现 9 个活跃设备块。

8.2 完整 Tag ID 图谱

表 4: 活跃 Tag ID 设备块全览

Block Base	典型值	特征	时效	推断设备
5000	1.875	全部等值（平衡三相）	实时	02204060029
5500	0-8.365	混合通道（变压器后备）	实时	02204060071
7500	2.54-3.69	中等负载	实时	02204060086
8000	2.87-3.69	中等负载	实时	02204060092
9000	1.87-2.38	平衡负载	实时	02204060100
9500	1.875	全部等值（平衡三相）	实时	02204060111
10000	2.98-3.04	历史快照	旧 (7/11)	02204060117
10500	2.94	单通道活跃	实时	02204060121
11000	1.99	单通道活跃	实时	02204060123

8.3 空间规律

1. 分配粒度：每设备 ~500 个 ID 槽位，实际使用 11-20 个连续 ID
2. 空白区：6000-7000（3 个块位），8500-8900（1 个块位），11500+（设备下线）
3. 设备 → 块映射：Tag Block Base ÷ 500 约对应 IOMan 设备列表中第 N 个设备

4. 数据时效分层：5000-9500 实时，10000 两天前，11500+ 半年前
5. 值分布特征：全等值 = 平衡三相设备（断路器），混合值 = 多通道设备（后备保护）

8.4 不依赖 Oracle 的发现路径

本次 Tag ID 扫描完全绕过了 Oracle 数据库。仅需 pSpace 连接凭据(admin/DQYTA11_pass) 和 ReadList 接口，即可完成全作业区点位发现。详见《OPC DA 点位自动发现方法论》。

9 结论与展望

9.1 主要成果

1. 成功逆向 psAPISDK.dll 的 Connect→ReadList 完整调用链
2. 确认 PS_DATA 结构体内存布局（24 字节对齐）
3. 实现自主 Tag ID 实时数据读取（python ctypes）
4. 完成 pSpace Tag ID 空间初步探索
5. 发现多处安全隐患（明文凭据、无加密协议）

9.2 待完成工作

1. Oracle SYS_POINTRELATION 表 → Tag ID 映射
2. pSpace Wire Protocol 完整帧格式（需更深层逆向）
3. psAPI_Real_NewSubscribeAndRead 回调机制测试
4. 本机 pSpace mock 服务与真实 IoMonitor 对接